column-hse-z: implementation

Technical notes on `run.mode = "column-hse-z"` (alias `column_hse`). HSEMHS, 24 Oct 2026.

1. Running it

```
[mpiexec -n R] julia --project=. -t T scripts/run.jl config/modes/column-hse-z.toml
```

`scripts/run.jl` parses the TOML (`Config.jl`: every key is declared with its default; unknown keys are rejected) and `run_config` dispatches to `run_column_hse` (`src/RunColumnHSE.jl`).

Key	Default	Meaning
`input.file`	required	NetCDF4/HDF5 cube: `x`, `y`, `z`, `temperature` (+ `B_x` etc. copied)
`input.crop_z_m`	none	keep levels inside $[z_{\rm top}, z_{\rm bottom}]$
`physics.g`	`"from_input"`	number or the file attribute `gravity_m_s2`
`eos.table_file`	`cache/eos_table_fullz.h5`	Wittmann EOS + 500 nm opacity table
`column_hse.anchor`	`bottom`	`bottom` | `top` | `height`
`column_hse.P_ref`	required	Pa, or `"from_input"` (attribute `P_ref_Pa`)
`column_hse.z_ref_m`	0	anchor height for `height`
`column_hse.n_sub`, `adapt`, `rk_tol`, `n_sub_max`	4, true, 10^{-6}, 64	RK4 substeps and doubling
`tau.top`, `H_max_m`, `mu`	`exp_tail`, 5×10^6, 1	τ top closure and viewing angle
`parallel.px`, `py`	0, 0	process grid (0: `MPI.Dims_create`)
`output.compression`	4	deflate level of the output datasets

2. Input

`open_input` (`IOInput.jl`) validates the axes (`z` strictly decreasing, `x`, `y` increasing, periodicity from the global attributes `periodic_x/_y`, `period_x_m/_y_m` or the FITS header cards `PERIOD_X/Y`) and identifies dimensions from each dataset's `DIMENSION_LIST`; units come from the `units` attribute (`m/km/cm/Mm`, `K`, `T/G`), never guessed. `read_field` reads only the rank's (x, y) hyperslab into a haloed `Field` (`OffsetArray`, indices $0{:}n_x{+}1$, $0{:}n_y{+}1$, $1{:}n_z$, Julia order (x, y, z), z top first) and checks $T > 0$, finite.

3. Algorithm

Per column (`ColumnHSE.jl`):

- `hse_column!`: from the anchor node ($k = N_z$ for `bottom`) the ODE $du/dz = -g\, e^{\ln\rho(X,u)-u}$, $dm/dz = -e^{\ln\rho}$ is integrated interval by interval; $X = \ln T$ linear in z inside an interval (`xlin`, exact at the nodes).
- `rk4_seg`: classical RK4 with n equal substeps, the EOS called at every stage (`lnrho`: bicubic Hermite table, status-returning).
- `rk4_adaptive`: the interval is run with n and $2n$ substeps; while $|u_{2n} - u_n| >$ `rk_tol` the count is doubled up to `n_sub_max`. Column flags record refinement (`COL_RK_REFINED`), tolerance not met, and EOS envelope exits (outputs NaN for that column).
- At the nodes, `state` gives $\ln\rho$, $\ln n_e$, $\ln\chi_{500}$; $P_e = n_e k_B T$, $n_{\rm H} = \rho/m_{\rm H,mix}$, $\bar\mu$, column mass (RK4 co-integrated and trapezoid).
- `tau_column!` (`Tau.jl`): top-first trapezoid of χ; `exp_tail` closure $\tau_{\rm top} = \chi_1 H_\chi$ if $\chi_2 > \chi_1$ and $0 < H_\chi < H_{\rm max}$, else half cell; method per column. `z_tau1`: crossing of $\log\tau = 0$, linear in $\log\tau$.
- `column_residual`: diagnostic $R_x = \partial_x P$, $R_y = \partial_y P$, $R_z = \partial_z P + \rho g$ with the collocated second-order operators of `Operators.jl` (halo exchange of P first).

4. Parallelisation

- MPI: 2-D Cartesian decomposition in (x, y), full columns on every rank (`Parallel.jl`, `THREAD_FUNNELED`, all MPI calls on the main task). Columns are independent, so no communication enters the values; only the residual diagnostic needs one halo exchange.
- Threads: owned columns are spread over `8 nthreads` chunks with a static schedule.
- The result is **bitwise** independent of the thread and rank counts (Gate 2 test "thread and rank invariance (bitwise)"). Measured: 1 vs 4 ranks identical on a 96^2 crop.
- Layer quantiles for the diagnostics tables gather each layer to rank 0 (`gather_layer`).

5. Output

HDF5 without parallel HDF5: `create_output` on rank 0, then `write_fields` writes the ranks' hyperslabs in a token ring. Groups:

- `/input`: copies of `temperature`, `B_x`, `B_y`, `B_z`, `x`, `y`, `z`;
- `/output`: `Pgas`, `rho`, `ne`, `Pe`, `nH_total`, `mu_bar`, `colmass`, `colmass_trap`, `chi500`, `tau500`, `logtau500` (h5py shape (x, y, z) like the input, i.e. Julia (z, y, x); each with `units`);
- `/diagnostics`: `R_x`, `R_y`, `R_z`, `eos_status`, `column_status`, `tau_top_method`, `rk_max_substeps`, `z_tau500_1`, per-layer tables `layer_pressure` and `layer_residual`, `z`;
- `/meta`: anchor, integrator, τ convention, EOS hash, config text and hash, git revision, layout.

Exit status: 0 when every column was integrated, 3 (`eos_failure`) when any column left the EOS envelope (its outputs are NaN and its `column_status` bit is set), 4 invalid input. The integration is direct, so there is no iterative convergence question; check `column_status` (refinement, tolerance not met) and `tau_top_method`.

6. Verification and performance

- Gate 2 (`test/test_gate2.jl`): isothermal ideal gas to 10^{-12}; $\ln T$-linear ideal gas against the analytic solution, fourth order under substep halving; τ closure and second order; Wittmann table on Bifrost columns; bitwise thread/rank invariance.
- Full production cube ($504 \times 504 \times 177$), 24 Oct 2026, commit 13666b7, 8 ranks $\times$ 32 threads on 4 nodes: 13 s to read, integrate and diagnose, 4 min to write the compressed output (11 fields plus diagnostics through the token ring); 2.3 GB per rank. 22 127 columns used substep refinement, none missed the tolerance. Log: `out/modes/column-hse-z.log` in the cluster worktree `MHS_modes` (not in git).
- The bitwise invariance holds because no reduction enters the column values; it is a property of this mode, not of the iterative modes.

7. Code map

`src/RunColumnHSE.jl` (driver, diagnostics tables, metadata), `src/ColumnHSE.jl` (integrator, residual), `src/Tau.jl`, `src/IOInput.jl`, `src/IOOutput.jl`, `src/Parallel.jl`, `src/Fields.jl`, `src/eos/Table.jl` (table interpolation), `config/modes/column-hse-z.toml`.